

Grado en Ingeniería Software

Tema 1
Concepto de Arquitectura Software y Principios de Diseño

Índice de la Presentación

- Concepto de Arquitectura Software
- Principios de Diseño Software
- Conclusiones

Concepto de Arquitectura Software

- Una **Arquitectura Software (AS)** representa la versión moderna del diseño de un sistema software.
- Las arquitecturas representan el diseño de un sistema software complejo desde distintos puntos de vista, según los intereses de diversos usuarios (“stakeholders”).

Concepto de Arquitectura Software

- Definición de Arquitectura Software (Perry & Wolf, 1991).
- **AS = {componentes, forma, lógica}**



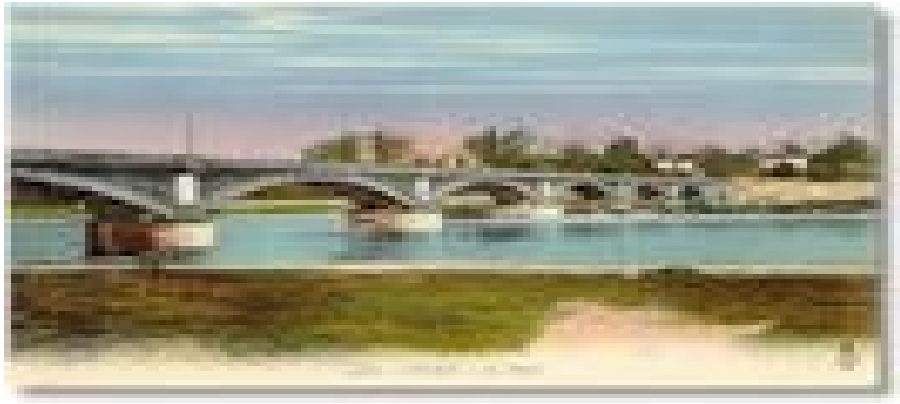
Una arquitectura software es una tripleta de elementos arquitectónicos como clases y paquetes UML que se conectan con una determinada forma. Diferentes arquitecturas pueden presentar formas diferentes (formas de organizar dichos componentes)
La lógica hace referencia a las decisiones que se toman para seleccionar un determinado componente o un patrón de diseño

Concepto de Arquitectura Software

Distintos tipos de arquitecturas



Diferentes Estilos

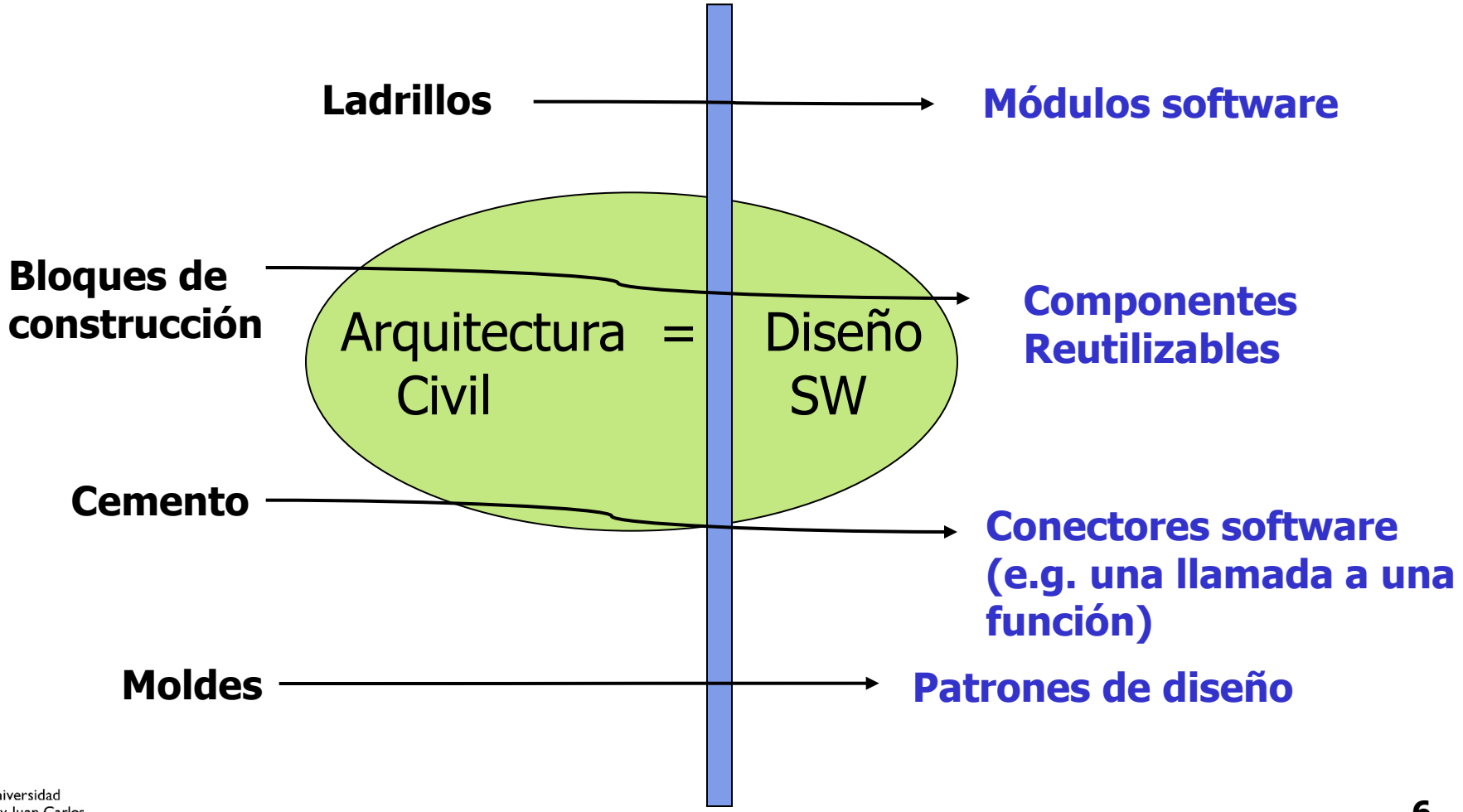




Son arquitecturas distintas con diferentes estilos. Cada una sirve para una misión diferente. Los componentes son distintos en algunos casos y se unen de manera diferente.

Concepto de Arquitectura Software

Es una analogía comparando la arquitectura de casas y construcciones civiles con la arquitectura software

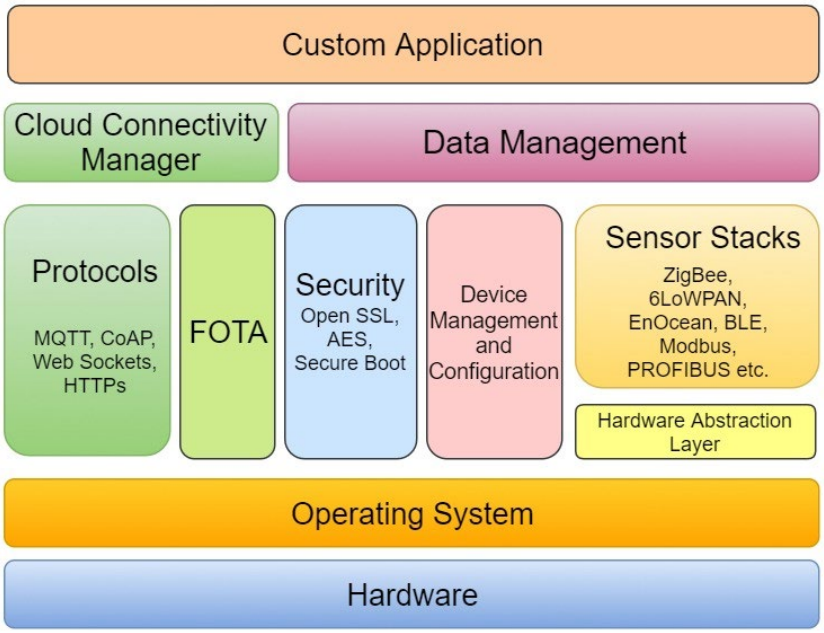


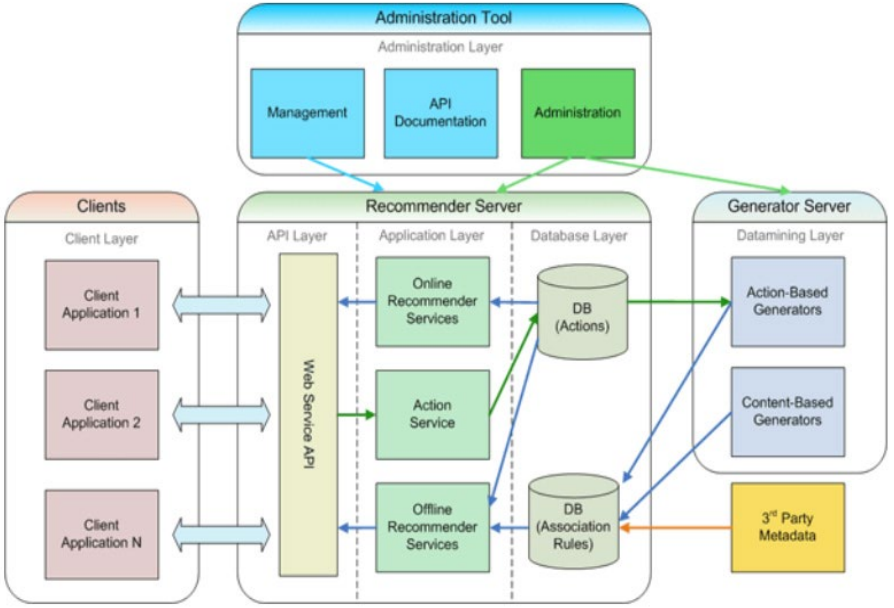
Concepto de Arquitectura Software

- Los diseñadores actuales se denominan Arquitectos Software
- La disciplina de la Arquitecturas Software tiene como misión identificar los requisitos funcionales y no funcionales que permitan construir el diseño software de un sistema
- Las Arquitecturas Software son la piedra angular de toda fase de diseño software

Concepto de Arquitectura Software

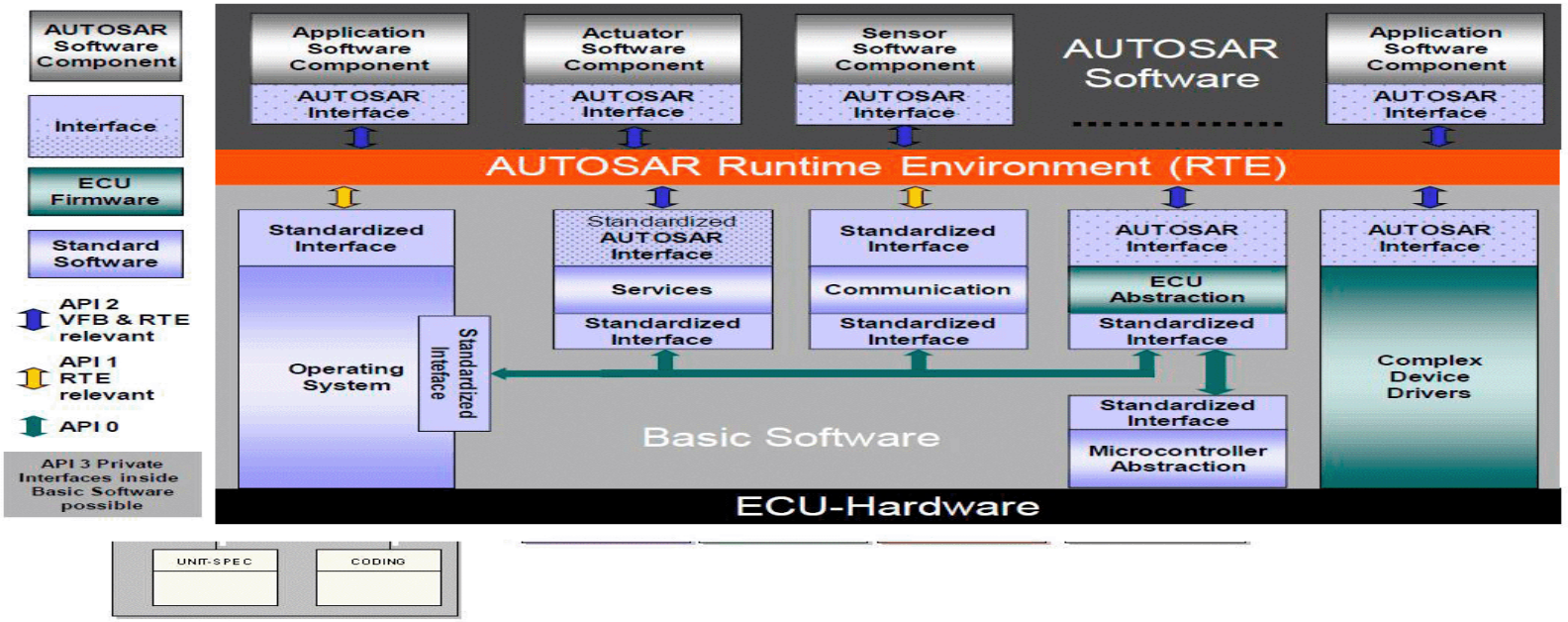
- Las arquitecturas software deben estar siempre alineadas con el código o implementación existente para evitar lo que se conoce como "*architectural mismatch*"
- Las arquitecturas software se "erosionan" o degradan (pérdida de calidad) durante las fases de evolución del sistema
- Arquitecturas software con diferente grado de detalle





Concepto de Arquitectura Software

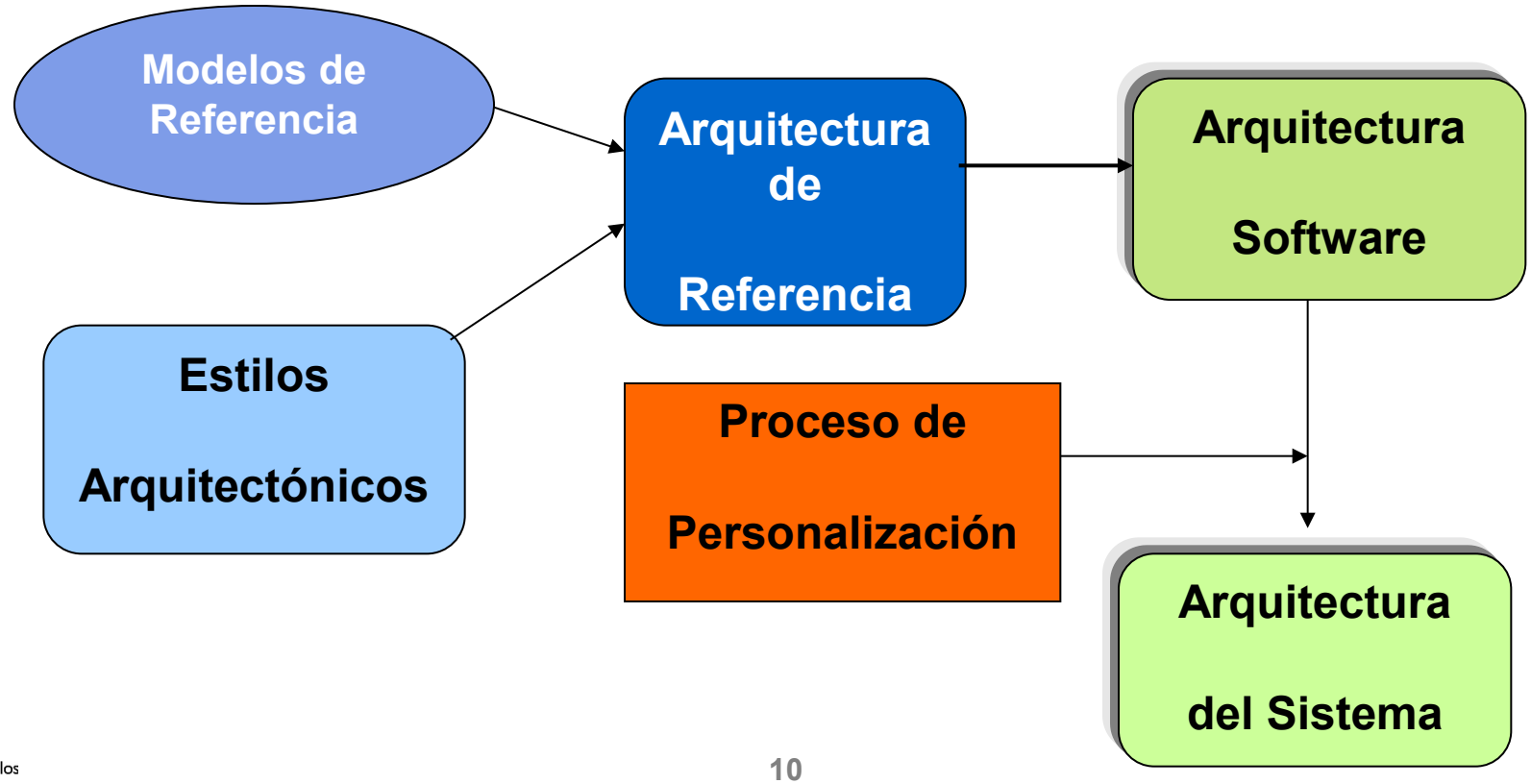
- Las Arquitecturas de Referencia son diseños software de alto nivel que describen las partes fundamentales de un conjunto de sistemas en un dominio particular
- E.g., Partes estándar de un compilador, Modelo OSI TCP/IP, arquitecturas en automoción (AUTOSAR), etc.



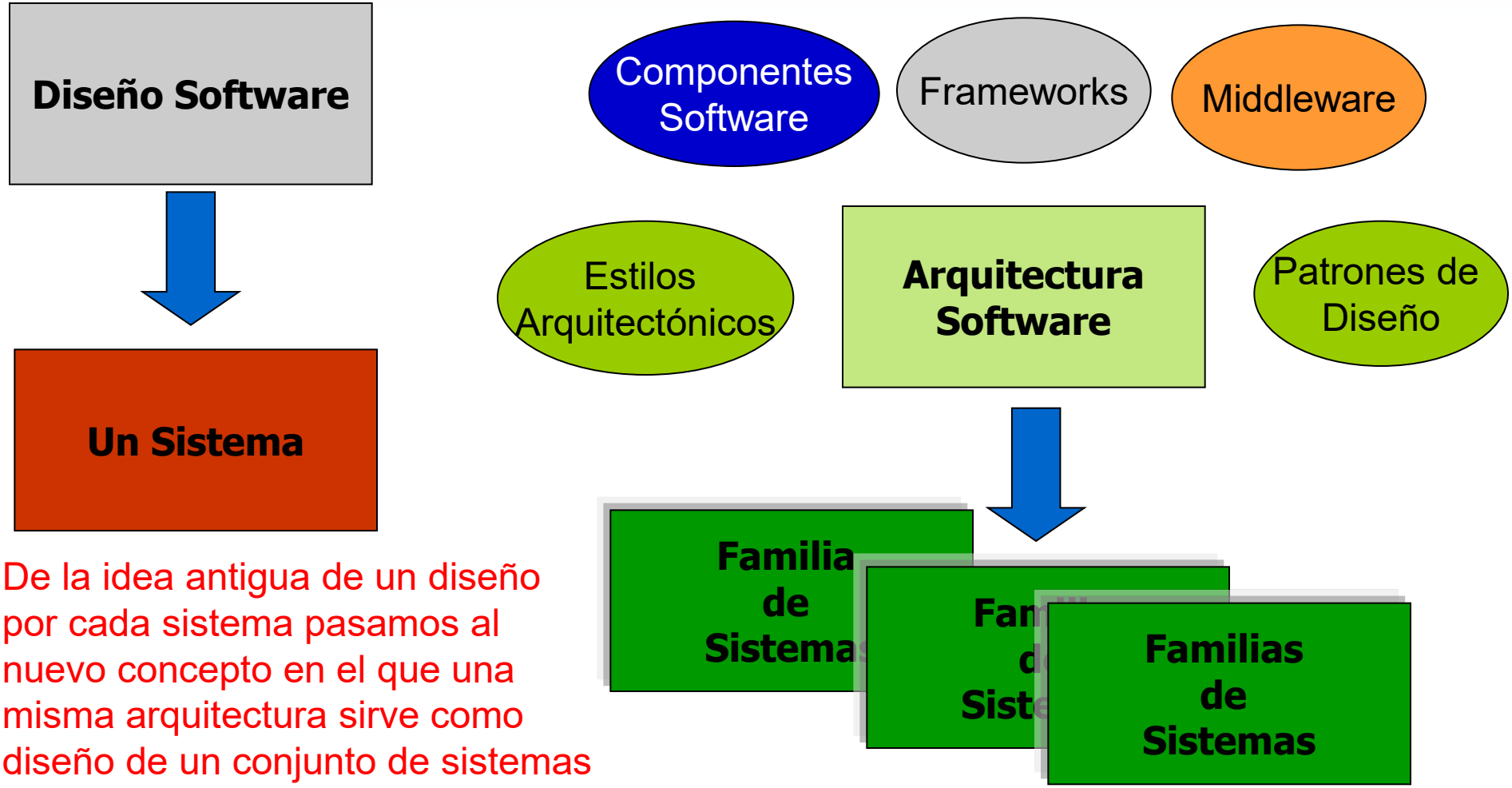
Concepto de Arquitectura Software

Modelos de Referencia: son una división de funcionalidad con un flujo de datos entre las partes que lo componen (ej: partes estándar de un compilador).

Arquitecturas de Referencia: son la asociación de un modelo de referencia sobre los componentes de software Y los flujos de datos entre dichos componentes.



Concepto de Arquitectura Software

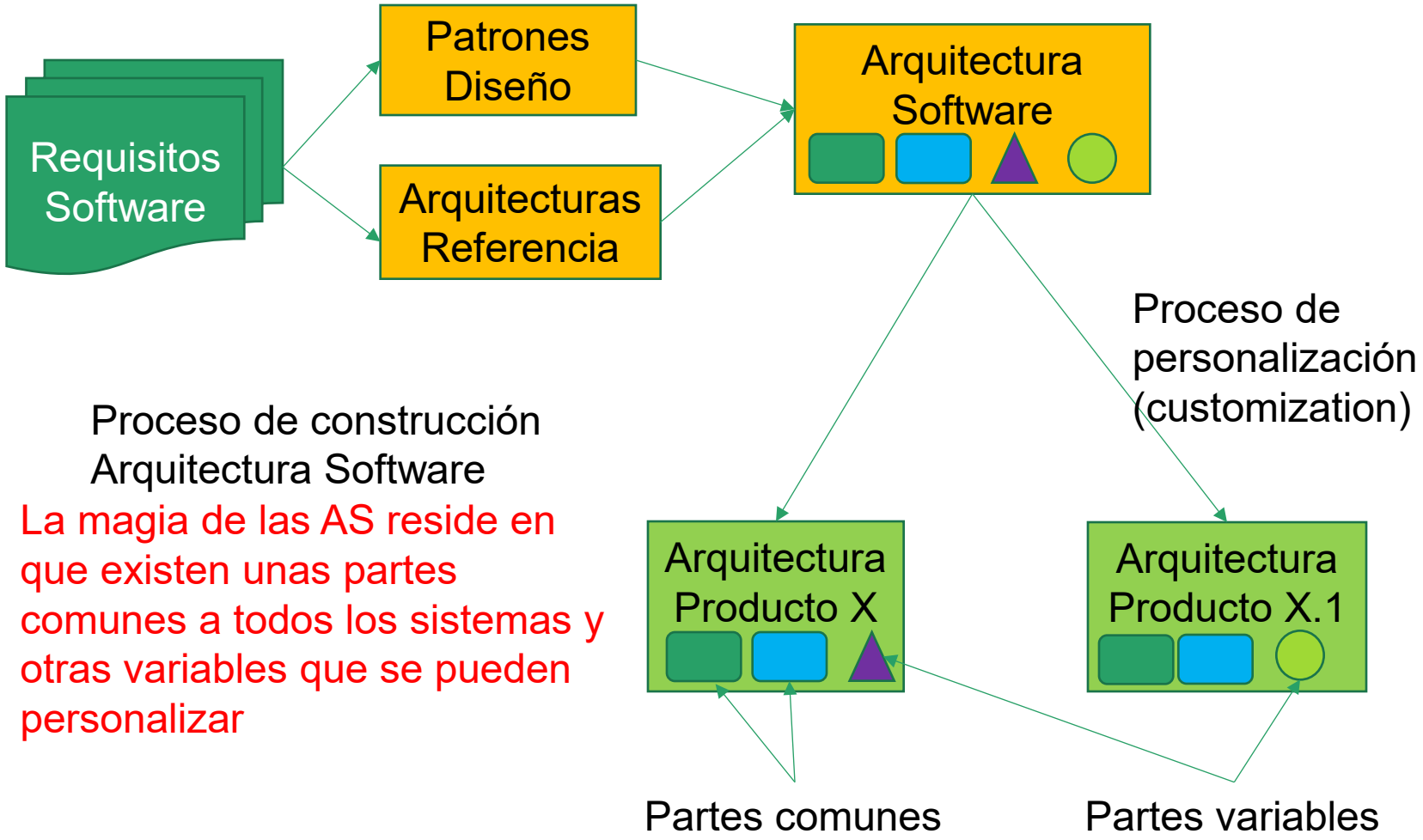


De la idea antigua de un diseño por cada sistema pasamos al nuevo concepto en el que una misma arquitectura sirve como diseño de un conjunto de sistemas similares

Concepto de Arquitectura Software

- La Arquitecturas del Producto Software es una personalización (derivación) de una arquitectura software para un producto determinado
- Se basa en la personalización de partes variables ("**software variability**") y el empleo de partes comunes reutilizables
- La arquitectura de un producto no se construye generalmente desde cero
- Refleja las características del producto concreto

Concepto de Arquitectura Software



La magia de las AS reside en que existen unas partes comunes a todos los sistemas y otras variables que se pueden personalizar

Principios de Diseño Software

- **Separación de responsabilidades** (“separation of concerns”): Evitar solapamientos de funcionalidad
- Hacer explícita la comunicación entre niveles
- Realizar abstracciones para diseñar niveles con **bajo grado de acoplamiento** (“loose coupling”)
- **Modularizar** el diseño y descomponer en subsistemas y módulos
- Las arquitecturas se rigen por un conjunto de **principios de diseño** software que no se deben violar

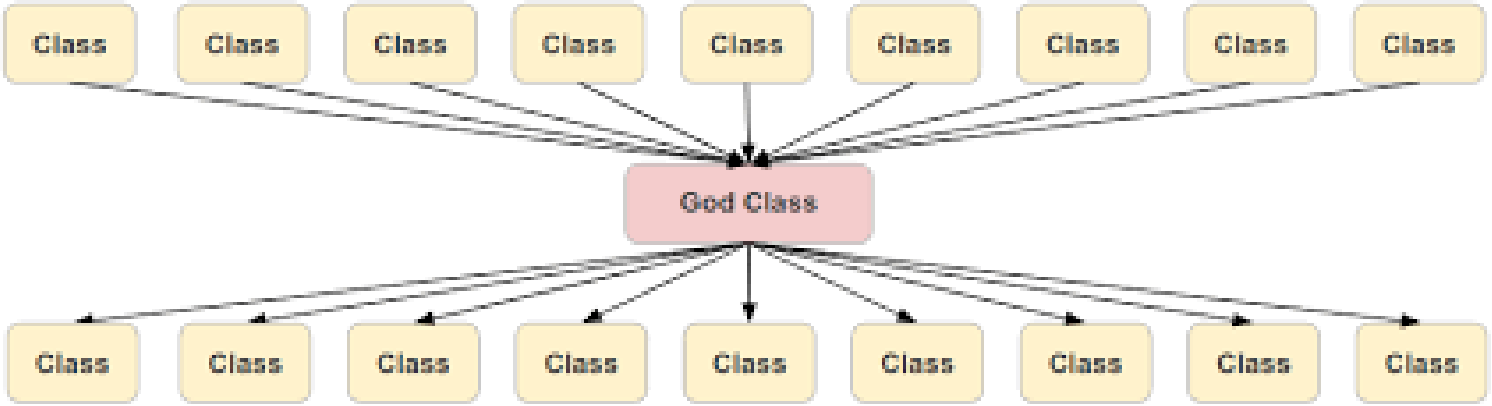
Principios de Diseño Software

Cohesión y acoplamiento son principios de diseño básico en todo sistema software para modularizar el código

- **[Bajo] Acoplamiento ("Coupling"):** Dependencias entre bloques o módulos de una arquitectura.
- **[Alta] Cohesión ("Cohesion"):** Dependencias entre variables y métodos dentro de un componente o módulo.

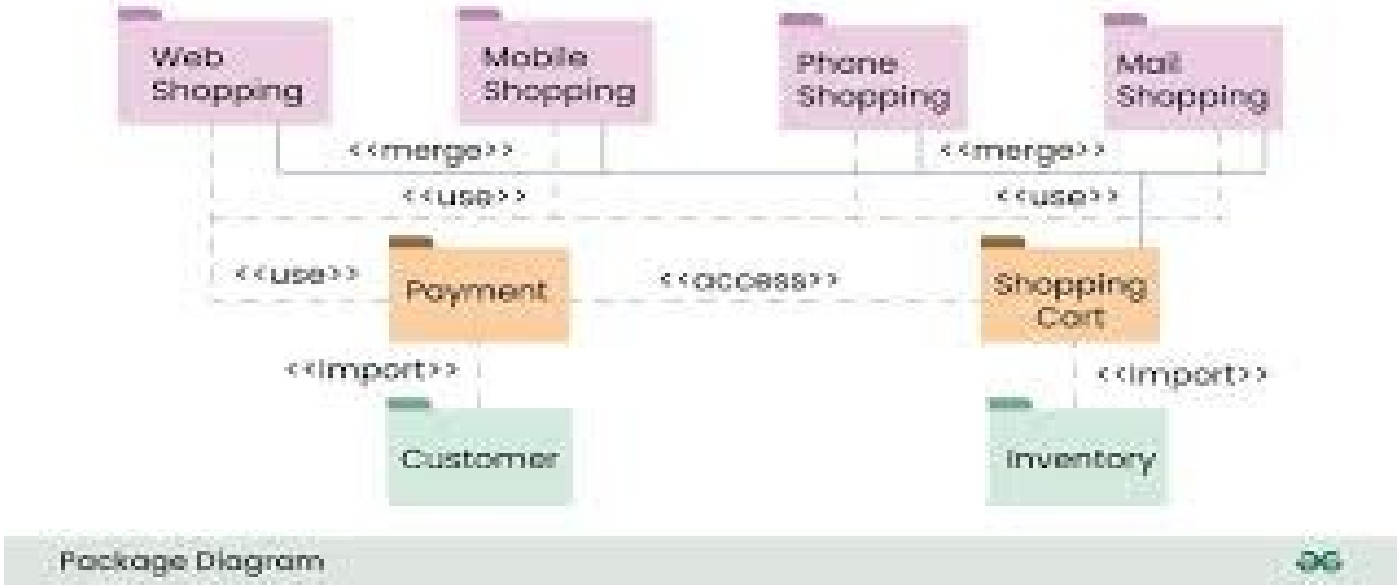
Principios de Diseño Software

- No duplicar la funcionalidad de componentes
- No sobrecargar (“overload”) la funcionalidad de componentes (**anti-patrón God Class**)
  - Difícil de mantener
  - Alta complejidad
  - Baja cohesión



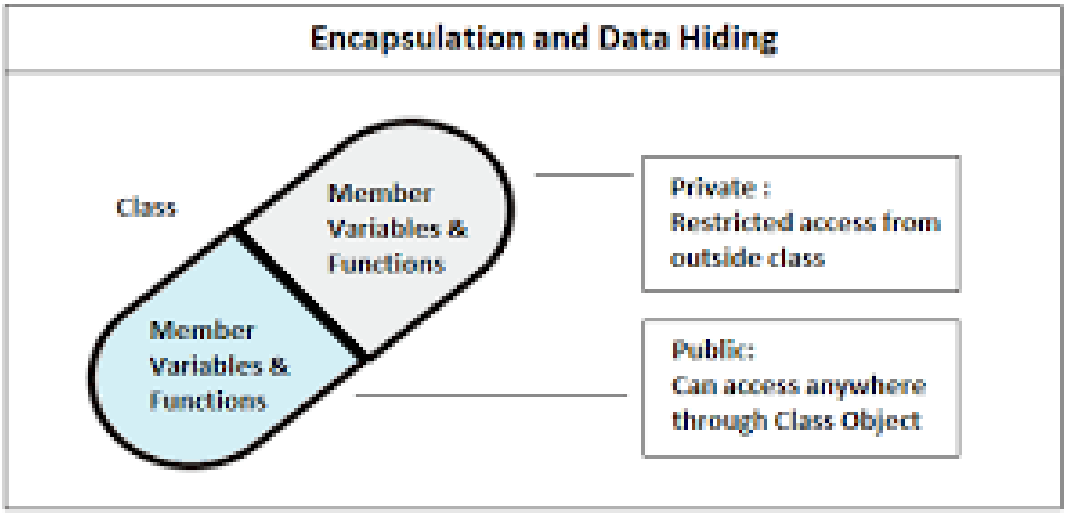
Principios de Diseño Software

- **Modularizar el diseño software y separación de responsabilidades** (paquetes UML corresponde a sistemas y subsistemas)



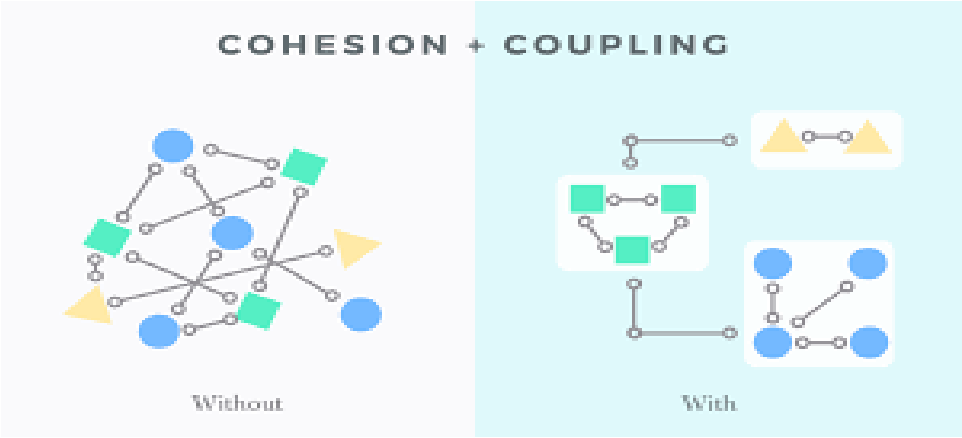
Principios de Diseño Software

- Un componente no tiene porqué conocer los detalles internos de otro componente (“**information hiding**”). Solamente se expone un interfaz público. Se reduce el acoplamiento

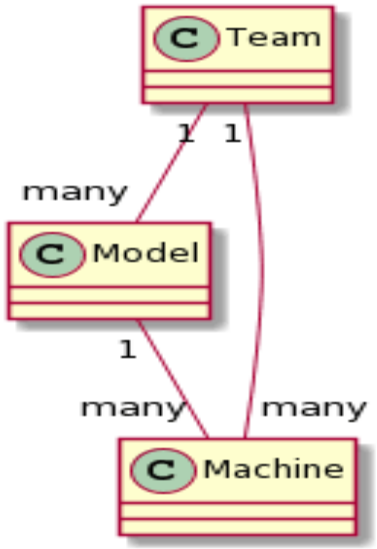


Principios de Diseño Software

- **Alta cohesión→Bajo acoplamiento:** El bajo acoplamiento se consigue mediante la abstracción, separación de responsabilidades y ocultamiento de la información



- **Evitar dependencias circulares**
  - **Mayor inestabilidad**
  - **Ripple effect**



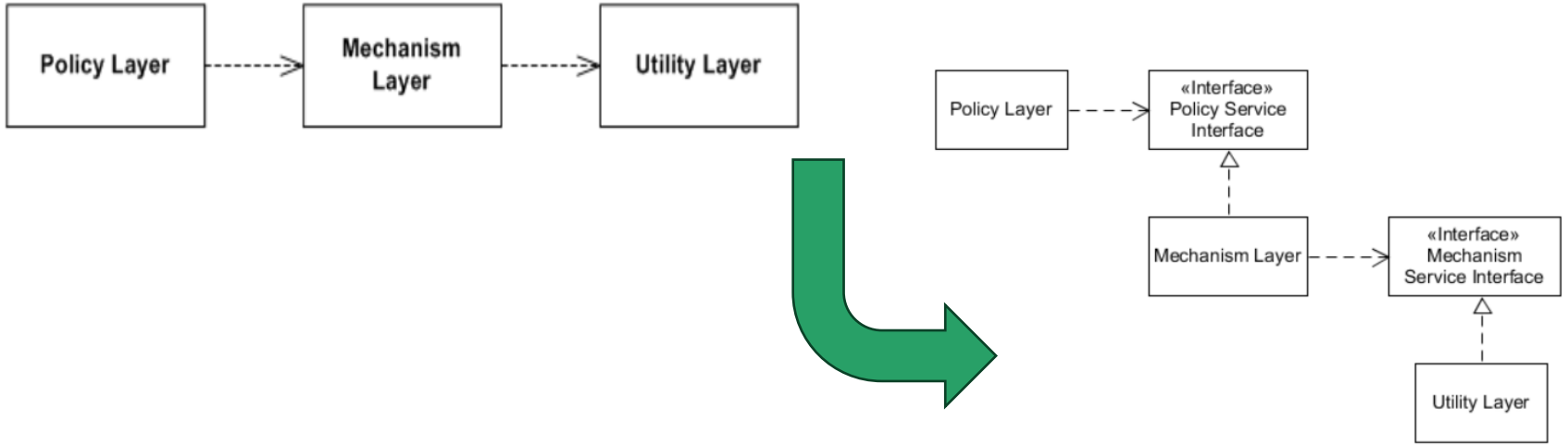
Principios de Diseño Software

Como conseguir "loose coupling"

- **Dependency Inversion Principle (DIP)**: Es una regla de diseño que desacopla reglas de negocio de alto nivel de los detalles de su implementación (bajo acoplamiento).
- Los módulos de alto nivel no deben depender de los de bajo nivel.

Principios de Diseño Software

- **Dependency Inversion Pattern**: introduce interfaces para desacoplar la lógica de negocio de su implementación. Los niveles de implementación altos y bajos dependen de interfaces que exponen el comportamiento que se requiere.
- Esta arquitectura agrupa los componentes superiores y las abstracciones que definen los servicios inferiores.
- Las capas de nivel inferior se crean mediante la herencia de clases abstractas o interfaces.



Principios de Diseño Software

- **Dependency Inversion Pattern**:

```
constructor() {
  this.repository = new UserMongoDBRepository();
  this.mailer = new GmailMailer();
}

login(email: string) {
  const user = this.repository.getUser(email);

  if (user) {
    this.mailer.sendLoginEmail(email);
```

1

```
interface Mailer {
  send(email: string)
}
```

2

```
class GmailMailer implements Mailer {
  send(email: string) {
    // Acceso al api de GMAIL para enviar
  }
}
```

3

```
login(email: string) {
  const user = this.repository.find(email);

  if (user) {
    this.mailer.send(email);
```

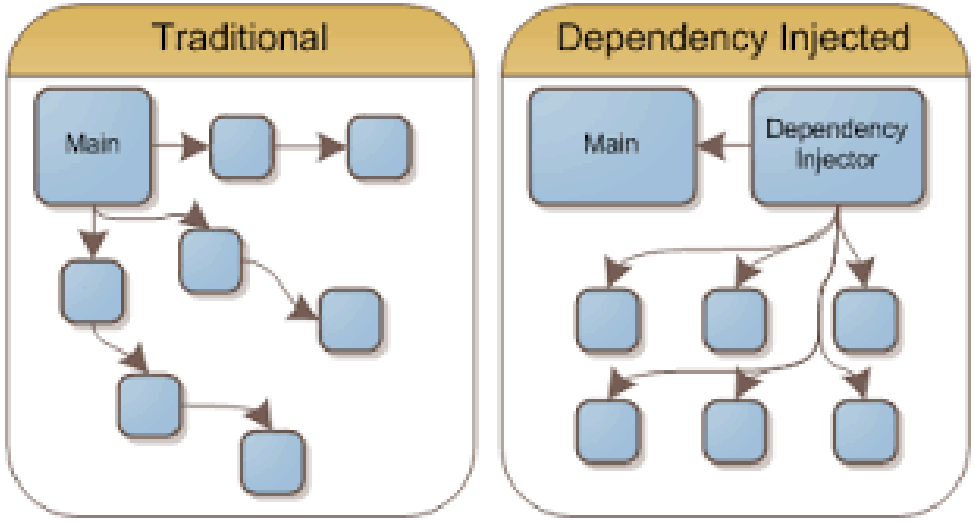
Principios de Diseño Software

- **Inversion of Control (IoC) principle**: Redirige el control a un manejador externo.
- **Principio de Hollywood**: “Don’t call us, we call you”
- No se invoca a las aplicaciones que se necesitan de manera directa. En su lugar un framework o un contenedor gestiona el control de flujo para el usuario.
- Proporciona bajo acoplamiento.
- Se implementa (entre otros) mediante **Dependency Injection**

Un ejemplo de IoC sucede cuando ponemos el ratón en una ventana de Windows, nos proporciona información sobre el programa que se ejecuta sin pedírselo al programa principal

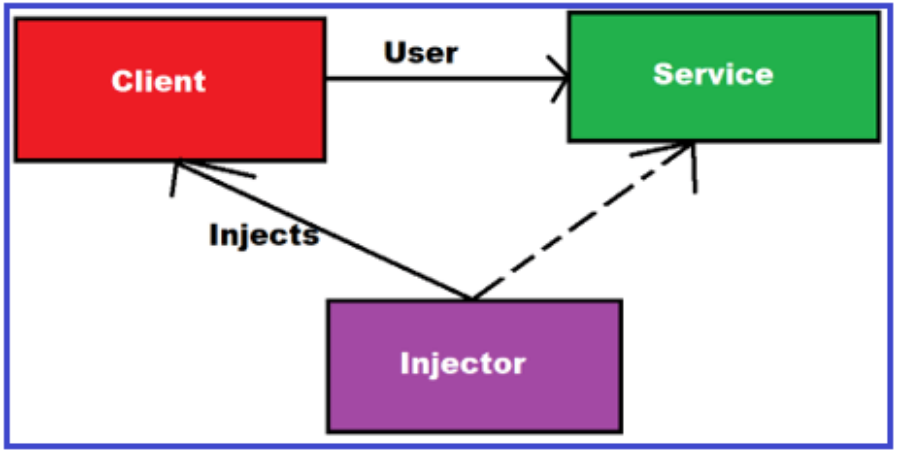
Principios de Diseño Software

- **Dependency Injection**: Un objeto recibe las dependencias que necesita en lugar de crearlas el mismo haciendo más modular el código.
- Facilita la separación de responsabilidades, el desacoplamiento y mejora el testing



Principios de Diseño Software

- **Dependency Injection**:
  - **Cliente**: Una clase o función que precisa de un servicio.
  - **Servicio**: Una herramienta o función que proporciona una característica
  - **Inyector**: Un código o framework que construye el servicio y lo pasa al cliente.
- El cliente no construye el servicio en el código
- Injector crea un objeto en “service” y lo injecta en la clase “client” el cual usa el objeto invectado para llamar a los métodos de service.



Conclusiones

- El proceso de construcción de arquitecturas software es *iterativo*, basado en la experiencia del arquitecto software, y utiliza patrones de diseño y estilos arquitectónicos.
- Las AS son una *combinación heterogénea de patrones y estilos* en los que suele predominar uno de ellos.
- Existen *Arquitecturas Específicas de Dominio* (automoción – AUTOSAR-, aviónica)
- Los principio de diseño software deben guiar la construcción de la arquitectura.

Lecturas adicionales

- Arquitectura de Referencia AUTOSAR https://www.autosar.org/
- Bredmeyer (Sotware Architecture) https://www.bredemeyer.com/
- Principios SOLID https://secture.com/solid-dependency-inversion-principle/
- Inversion of control / dependency injection http://stackoverflow.com/questions/6550700/inversion-of-control-vs-dependency-injection
- Microservices (Style and Patterns) https://microservices.io/
- DIP (Java): https://www.digitalocean.com/community/tutorials/java-dependency-injection-design-pattern-example-tutorial